AI-NATIVE VERSION CONTROL SYSTEM

Vault-AI

Complete A-Z Operating Manual

Merkle Trees • Ollama Local AI • Policy-as-Code

Shadow Storage • High-Trust Merging • Tamper-Proof History

Phase 1-9 Complete | 22 Commands | Zero-Cost Local AI | Production-Ready

April 2026

Print / Save as PDF

> ***Vault-AI*** *is an AI-native version control system built around a* ***"no staging area"*** *philosophy. Every commit is a direct atomic snapshot, secured by cryptographic Merkle-tree hashing, guarded by AI-powered security modules, and managed through a human-readable CLI.*

Table of Contents

1. Installation & Setup

2. Quick Start (5-minute workflow)

3. Core Concepts

4. All Commands Reference (A-Z)

- vault init

- vault save

- vault diff

- vault history

- vault undo

- vault scan

- vault story

- vault predict

- vault merge

- vault map

- vault search

- vault sandbox

- vault hydrate

- vault watch
- vault create branch
- vault snapshot
- vault sync
- vault env show
- vault env diff
- vault config list-ai
- vault config use
- vault config use-custom
- vault config reset

5. Configuration Files
6. AI Engine Setup
7. Security & Policy System
8. Object Lineage & Dataset Tracking
9. Audit Logs
10. Troubleshooting

---

1. Installation & Setup

Requirements

- Python 3.9+**
- Ollama** (local AI engine — recommended): https://ollama.ai
- Optional: `flake8` or `eslint` for self-healing lint checks

Project Structure

```
your-project/
├── .vault/                  ← Vault-AI internal store (auto-created)
│   ├── HEAD                 ← Current commit pointer
│   ├── refs/heads/main      ← Branch head
│   ├── objects/             ← Merkle tree object store
│   │   └── large/           ← Shadow Storage for >50MB files
│   ├── metadata/
│   │   └── lineage.json     ← AI model/dataset hashes
│   ├── config.json          ← AI engine configuration
│   └── .audit_log           ← AI transparency trail
├── .vault_policy            ← (optional) Policy rules
├── .vault-secrets-allow     ← (optional) Secret allowlist
├── .vault_private           ← Auto-created quarantine log
└── your-code-files/
```

---

2. Quick Start

```
# Navigate to your project
cd /path/to/your-project


# 1. Initialize the repo
```

```
python3 /path/to/VCS/cli.py init


# 2. Write some code, then commit
echo "print('hello')" > app.py
python3 /path/to/VCS/cli.py save -m "First commit"


# 3. See what changed
python3 /path/to/VCS/cli.py diff


# 4. View history
python3 /path/to/VCS/cli.py history


# 5. Undo last commit
python3 /path/to/VCS/cli.py undo last
```

> ***Tip:*** *Add an alias in your shell profile:*
>
> *` bash*
>
> *alias vault="python3 /Users/sathvikbadam/Desktop/VCS/cli.py"*
>
> *`*
>
> *Then use `vault save`, `vault diff`, etc. directly.*

---

3. Core Concepts

Concept	How It Works
No Staging Area	`vault save` commits *everything* in one shot — no `git add` needed
Merkle Tree	Every file is SHA-256 hashed into a tree; history is tamper-proof
Shadow Storage	Files >50MB are auto-offloaded to `.vault/objects/large/` via `.vlink`
Default-to-Local AI	Ollama runs on `localhost:11434` — no API keys, no cost
Policy Guard	`.vault_policy` blocks commits that violate your rules
High-Trust Merge	AI stops and asks YOU if confidence < 90% before merging

4. All Commands Reference

`vault init`

Purpose: Initialize a new Vault-AI repository in the current directory.

```
vault init
```

Creates the `.vault/` directory with internal structure (`HEAD` , `refs/` , `objects/` , metadata).

> *Run this once per project, similar to `git init` .*

`vault save`

Purpose: Commit all current changes to the repository. No staging required.

```
vault save -m "Your commit message"
vault save                          # AI generates the commit message
vault save -m "msg" --force         # Bypass security checks
```

Flags:

Flag	Description
`-m "message"`	Commit message. If omitted, AI auto-generates one from the diff
`--force`	Bypass Secret Guard, Anomaly Detector, and Policy checks

What happens on every save:

1. 🔐 **Secret Guard** — scans for hardcoded credentials
2. 🔧 **Self-Healing Lint** — checks Python/JS for syntax errors
3. 🚨 **AI Anomaly Detector** — checks for eval(), network calls, reverse shells
4. 📋 **Policy-as-Code Validator** — enforces `.vault_policy` rules

5. 🌳 **Merkle Tree Snapshot** — hashes all files into the object store
6. 🧬 **Object Lineage** — auto-tracks `.bin`, `.onnx`, `.csv`, `.pth` files
7. ⏱️ **Cycle-Time Print** — shows how fast the commit took (ms)

Example output:

```
✦ [main a3f9c1d0b2] Initial commit

⏱ Cycle-Time Benchmark: Vault-AI finalized snapshot in 21.04ms
```

`vault diff`

Purpose: Show a semantic diff of the current working tree vs the last commit.

```
vault diff

vault diff -v        # verbose (more detail)
```

Displays files changed since the last commit with line-level additions/deletions.

`vault history`

Purpose: Show the commit log for the current branch.

```
vault history

vault history -n 10   # Show last 10 commits only (default 20)

vault history --limit 50
```

Example output:

```
  a3f9c1d0b2  2026-04-17 23:00  Initial commit

  7f2ae89123  2026-04-17 23:05  Add API layer

  cc89ab1210  2026-04-17 23:10  Fix bug in parser
```

`vault undo`

Purpose: Revert the last committed snapshot, restoring the prior state.

```
vault undo last           # Undo the most recent commit

vault undo last --sparse  # Undo in sparse (lazy) mode
```

Flags:

Flag	Description
`last`	Required target — currently only `last` is supported
`--sparse`	Restore files via lazy-load (doesn't extract full tree to disk)

Example output:

```
⏪ Undid commit a3f9c1d0b2. Now at [main 7f2ae89123]
   Restored 3 files from parent commit.
```

`vault scan`

Purpose: Run a full security audit without committing.

```
vault scan
```

Executes three security checks in sequence:

1. **Integrity Verification** — Validates the Merkle chain; locks repo if tampered
2. **Dependency Audit** — Checks `requirements.txt` / `package.json` against OSV.dev CVE database
3. **AI Anomaly Detection** — Scans working tree for suspicious logic patterns

Example output:

```
🔍 Initializing Security Scan...
✅ Integrity: Merkle chain is clean — history has not been tampered with.
🔎 Auditing 3 Python packages against OSV.dev...
✅ System looks clean and secure!
```

> *If tampering is detected: `.vault/LOCKDOWN` is created and all further commits are blocked.*

`vault story`

Purpose: Generate a human-readable narrative summary of recent development activity.

```
vault story                  # Last 7 days (default)
vault story --days 30        # Last 30 days
vault story --pdf            # Export story as a PDF file
```

Flags:

Flag	Description
`--days N`	Number of days to include (default: 7)
`--pdf`	Export the generated story as a PDF document

Example output:

```
## Development Story (Last 7 Days)

**12 commits** from 2026-04-10 to 2026-04-17

### Commit Log
- `a3f9c1d0b2` 2026-04-17 — Added auth module
- `7f2ae89123` 2026-04-17 — Fixed token expiry bug
```

`vault predict`

Purpose: Predict merge conflicts between two branches *before* actually merging.

```
vault predict main feature-x
vault predict branch_a branch_b
```

Arguments:

Argument	Description
`branch_a`	The first branch name
`branch_b`	The second branch name

Uses AI to analyze logic dependencies and detect files that will conflict.

Example output:

```
⚠ Predicted Conflicts: 2 file(s)
  app.py      — Modified in both branches
  config.py   — Logic dependency on 'auth' module
```

`vault merge`

Purpose: Merge a branch into the current one using the AI Judge.

```
vault merge feature-x
vault merge hotfix-branch
```

How it works:

1. Predicts conflicts and identifies file differences
2. Sends conflicting file pairs to the AI Judge
3. **If confidence ≥ 90%** — auto-resolves and commits the merge
4. **If confidence < 90%** — pauses and shows reasoning for manual review:

```
🛡 High-Trust Merge Pause:
```

```
      'I merged these using semantic logic because the contexts overlap,
       but I am only 82% sure. Please review.'
      Accept AI merge proposal? [y/N]:
```

5. Non-conflicting files from the source branch are carried over automatically

---

`vault map`

Purpose: Display a visual ASCII branch map of commit history.

```
vault map
vault map -n 50      # Show last 50 commits
vault map --limit 10
```

Shows commits, branches, and their relationships in the terminal.

`vault search`

Purpose: Natural language search through commit history and content.

```
vault search "database connection"
vault search "login bug fix"
vault search "authentication"
```

How it works:

- If **ChromaDB** is installed → uses semantic vector similarity search
- If ChromaDB is absent → falls back to substring search across commit messages

> *Install ChromaDB for best results:* `pip install chromadb`

`vault sandbox`

Purpose: Enter an isolated "what-if" workspace to experiment with changes safely.

```
vault sandbox            # Enter sandbox (creates isolated copy)
vault sandbox status     # Check if you're inside a sandbox
vault sandbox exit       # Exit, discarding all sandbox changes
vault sandbox merge      # Exit, keeping sandbox changes in working tree
```

Use case: Try risky refactors, prototype features, or test destructive operations without polluting your real commit history.

`vault hydrate`

Purpose: Restore (extract) files from the lazy-load sparse manifest on demand.

```
vault hydrate .                    # Hydrate all sparse files
vault hydrate src/models/          # Hydrate files in a specific directory
vault hydrate model_weights.bin    # Hydrate a specific file
```

Used for **monorepo efficiency** — only extract parts of the repository you actually need to work on.

> *Large files (>50MB) are stored as `.vlink` metadata references. `hydrate` swaps them back for real files.*

`vault watch`

Purpose: Start a background file-watcher daemon that auto-saves the project at regular intervals.

```
vault watch                  # Default: auto-save every 30 seconds
vault watch -i 60            # Auto-save every 60 seconds
```

```
vault watch --interval 120   # Auto-save every 120 seconds
```

Flags:

Flag	Description
`-i N` / `--interval N`	Poll interval in seconds (default: 30)

The daemon watches for changes and creates automatic snapshots using the label `"auto-save-[timestamp]"` .

> *Press `Ctrl+C` to stop the daemon.*

`vault create branch`

Purpose: Create a new branch from the current HEAD.

```
vault create branch <name>

# Examples:
vault create branch feature-login
vault create branch hotfix-v2
```

`vault snapshot`

Purpose: Manually create a labeled time-machine snapshot of the current state.

```
vault snapshot                          # Creates snapshot labeled "manual"
vault snapshot -l "before-big-refactor" # Custom label
vault snapshot --label "pre-deploy"
```

Flags:

Flag	Description
`-l` / `--label`	A descriptive label for this snapshot (default: "manual")

Snapshots are separate from commits — they act as named checkpoints you can jump back to.

`vault sync`

Purpose: Push all local objects to a remote Vault server.

```
vault sync                        # Push to http://127.0.0.1:8000 (default)
```

```
vault sync --url http://myserver:8000
```

Flags:

Flag	Description
`--url URL`	Remote server URL (default: `http://127.0.0.1:8000`)

> *Run `python3 server.py` to start a local remote server for team collaboration.*

`vault env show`

Purpose: Display the environment snapshot captured at a specific commit.

```
vault env show                  # Show snapshot from current HEAD
vault env show a3f9c1d0b2       # Show snapshot from a specific commit SHA
```

Example output:

```
🌍 Environment Snapshot (commit a3f9c1d0b2)
────────────────────────────────────────
Python   : 3.11.4
OS       : Darwin 24.6.0 (arm64)
```

```
  Host     : MacBook-Air.local
  Packages : 3 installed
             numpy==1.25.0
             pandas==2.0.0
             requests==2.28.0
```

`vault env diff`

Purpose: Compare the *current* runtime environment against a stored snapshot.

```
vault env diff                  # Diff current vs HEAD snapshot
vault env diff a3f9c1d0b2       # Diff current vs specific commit
```

Example output:

```
🔄 Environment Diff vs stored snapshot:
────────────────────────────────────────

   Packages added (1):
    pandas==2.0.0

 - Packages removed (1):
```

```
    scipy==1.9.0


~ Packages changed (1):

    requests: 2.28.0 → 2.31.0
```

`vault config list-ai`

Purpose: List all available AI adapters and show the currently active one.

```
vault config list-ai
```

Example output:

```
🤖 Available AI Adapters:

    * Ollama (Local Default)  (Active)

      Gemini (Cloud)

      OpenAI (Cloud)

      Custom Script
```

`vault config use`

Purpose: Switch the active AI engine.

```
vault config use ollama     # Switch to local Ollama (default)
vault config use gemini     # Switch to Google Gemini (requires GEMINI_API_KEY)
vault config use openai     # Switch to OpenAI GPT (requires OPENAI_API_KEY)
```

`vault config use-custom`

Purpose: Wire a personal `.py` or `.sh` script as the AI "Brain."

```
vault config use-custom /path/to/my_agent.py
vault config use-custom /path/to/my_agent.sh
```

Requirements for custom script:

- Must be **executable** (`chmod +x script.py`)
- Must **read prompt from stdin**
- Must **write response to stdout**

Example custom agent (`my_agent.py`):

```
#!/usr/bin/env python3
import sys
prompt = sys.stdin.read()
print(f"Custom response to: {prompt[:50]}")
```

When JSON output is required, the env var `VAULT_REQUIRE_JSON=1` is injected automatically.

`vault config reset`

Purpose: Revert the AI engine back to the default local Ollama.

```
vault config reset
```

Example output:

```
✅ AI Config reset to default (Ollama).
```

---

5. Configuration Files

`.vault_policy` — Policy-as-Code

Create this file in your project root to enforce commit rules:

```
{
  "no_console_logs": true,
  "require_test_files": true
}
```

Rule	Type	Description
`no_console_logs`	boolean	Block commits containing `print()` or `console.log()`
`require_test_files`	boolean	Block commits if no `test_*.py` file exists

When a rule is violated:

```
🚨 POLICY-AS-CODE BREACH DETECTED:
   - app.py: contains print/console logger.
   - Policy requires at least one unit test script, but none found.
```

Override with: `vault save --force`

`.vault-secrets-allow` — Secret Allowlist

To whitelist an entire file from Secret Guard scanning:

```
config/dev_settings.py
fixtures/test_data.env
```

To suppress a single line, add `# vault-ok` at the end:

```
API_KEY = "test-key-123"  # vault-ok
```

`.vault/config.json` — AI Engine Config

Auto-managed by `vault config`. Manual structure:

```
{
  "agent": "ollama",
  "custom_path": null
```

```
}
```

---

6. AI Engine Setup

Option A: Ollama (Default — Free, Local, No API Key)

```
# Install Ollama
curl -fsSL https://ollama.ai/install.sh | sh

# Pull a model (recommended)
ollama pull deepseek-coder

# Start Ollama (runs automatically on macOS after install)
ollama serve
```

Environment variables (optional):

```
export OLLAMA_URL=http://localhost:11434    # default
export OLLAMA_MODEL=deepseek-coder         # model to use
```

Option B: Gemini (Free Tier Cloud)

```
export GEMINI_API_KEY=your_key_here
export GEMINI_MODEL=gemini-1.5-flash       # optional, this is default
vault config use gemini
```

Option C: OpenAI (GPT-4)

```
export OPENAI_API_KEY=sk-...your_key_here
export OPENAI_MODEL=gpt-4-turbo            # optional
vault config use openai
```

Option D: Custom Script

```
# Create your script
```

```
echo '#!/usr/bin/env python3
import sys
print("My AI says: " + sys.stdin.read())
' > my_agent.py
chmod +x my_agent.py


# Register it
vault config use-custom ./my_agent.py
```

---

7. Security & Policy System

How Secret Guard Works

Every `vault save` (without `--force`) scans all files for patterns matching:

- AWS/GCP/Azure access keys
- Hardcoded passwords and tokens
- Private keys (RSA, PEM blocks)
- High-entropy random strings

Detected secrets are **auto-quarantined** to `.vault_private` and the commit is blocked.

How the Tamper-Proof System Works

Every object in `.vault/objects/` is validated on every `vault scan` using zlib integrity checks. If **any** byte has been altered:

```
🔐 LOCKDOWN ENGAGED: Repository is locked.
```

`.vault/LOCKDOWN` is created. All future commits are blocked until the lockdown is manually resolved.

AI Anomaly Detection Patterns

Automatically scanned on every commit:

Pattern	Description
`eval()` / `exec()`	Dynamic code execution
`os.chmod` / `subprocess chmod`	Permission escalation
Hardcoded tokens/passwords	Credential injection
`base64.b64decode` + `_import_`	Obfuscated imports
`/dev/tcp` , `nc -e` , `bash -i >&`	Reverse shell patterns
`ctypes.CDLL` / `ctypes.windll`	Native DLL access
`shutil.rmtree('/')`	Destructive root deletion
Hidden network calls	Covert data exfiltration

8. Object Lineage & Dataset Tracking

Vault-AI automatically detects and SHA-256 fingerprints:

Extension	Type
`.bin`	Binary weights
`.onnx`	ONNX model files
`.pth` / `.pt`	PyTorch checkpoints
`.h5`	Keras/TensorFlow weights
`.csv` / `.jsonl`	Datasets
`.parquet`	Big data format

Hashes are stored in `.vault/metadata/lineage.json`:

```
[
  {
    "path": "models/bert_weights.bin",
    "sha256": "f88b7331...",
    "size_bytes": 438000000
  }
]
```

This guarantees **perfect reproducibility** of any historical state — including model weights and training data.

---

9. Audit Logs

Every AI action (commit messages, merge suggestions, anomaly checks) is recorded in `.vault/.audit_log` :

```
{"timestamp": 1716449656.66, "action": "AI Prompt Generated", "provider_model": "ollar
```

Field	Description
`timestamp`	Unix epoch of the action
`action`	What the AI was asked to do
`provider_model`	Which AI engine was used
`confidence`	Confidence score if available
`system_prompt`	The system context sent to the AI
`user_prompt`	The actual user query (first 500 chars)

---

10. Troubleshooting

"Not inside a Vault-AI repository"

You haven't run `vault init` in this directory.

```
vault init
```

"LOCKDOWN ENGAGED"

Your `.vault/objects/` were tampered with externally. Delete `.vault/LOCKDOWN` only if you're certain the corruption was accidental:

```
rm .vault/LOCKDOWN
```

Commit blocked by Secret Guard

Your file contains a credential pattern. Either:

- Remove the secret and use environment variables instead
- Add `# vault-ok` at the end of that specific line
- Use `vault save --force` to override (⚠ risky)

"ChromaDB not installed"

Semantic search falls back to substring search. For full vector search:

```
pip install chromadb
```

Ollama not responding

```
# Start Ollama
ollama serve

# Or check status
curl http://localhost:11434/api/tags
```

Custom agent script not working

- Ensure the script is executable: `chmod +x script.py`
- Ensure it reads from `stdin` and writes to `stdout`
- Test manually: `echo "test prompt" | ./script.py`

Vault-AI — matematically reproducible, architecturally transparent, and refuses to make high-stakes decisions without human trust.